

Deep Learning to LLMs

Prerequisites

- Python and NumPy (*search: “Python NumPy tutorial for beginners”*)
- Basic machine learning: train/validation/test split, overfitting and underfitting, loss and metrics, gradient descent (*search: “machine learning basics for beginners”*)
- Tools: Jupyter or Google Colab; Keras/TensorFlow or PyTorch (*search: “Google Colab tutorial”, “Keras tutorial for beginners”, “PyTorch tutorial for beginners”*)

Stage 1 — Neural network foundations

- Artificial neuron and perceptron (*search: “perceptron neural network explained”*)
- Perceptron learning rule and geometric interpretation (*search: “perceptron learning rule”*)
- Perceptron loss functions: hinge loss, binary cross-entropy, sigmoid (*search: “hinge loss”, “binary cross entropy explained”, “sigmoid function neural network”*)
- Limitations of a single neuron (*search: “XOR problem perceptron limitation”*)
- Multi-layer perceptron (MLP) (*search: “multilayer perceptron explained”*)
- MLP notation and forward propagation (*search: “forward propagation neural network”*)
- Projects: churn prediction; handwritten digit classification (MNIST); regression with an MLP (*search: “customer churn prediction neural network”, “MNIST digit classification Keras”*)

Stage 2 — How networks learn

- Loss functions for regression and classification (*search: “loss functions in deep learning”*)
- Backpropagation (*search: “backpropagation neural network”*)
- Gradient descent: batch, stochastic, mini-batch (*search: “batch vs stochastic vs mini batch gradient descent”*)
- Vanishing and exploding gradients (*search: “vanishing and exploding gradient problem”*)
- Regularization: early stopping, feature scaling, dropout, L1/L2 (*search: “early stopping neural network”, “feature scaling neural network”, “dropout layer explained”, “L1 L2 regularization deep learning”*)
- Activation functions: sigmoid, tanh, ReLU, Leaky ReLU, PReLU, ELU, SELU (*search: “activation functions explained”, “ReLU variants leaky relu prelu elu selu”*)
- Weight initialization: zero/constant, Xavier/Glorot, He (*search: “weight initialization neural network”, “Xavier and He initialization”*)
- Batch normalization (*search: “batch normalization explained”*)
- Optimizers: exponentially weighted averages, momentum, Nesterov accelerated gradient, AdaGrad, RMSProp, Adam (*search: “exponentially weighted moving average”, “SGD momentum optimizer”, “nesterov accelerated gradient”, “adagrad optimizer”, “rmsprop optimizer”, “adam optimizer explained”*)
- Hyperparameter tuning (*search: “Keras Tuner hyperparameter tuning”*)

Stage 3 — Convolutional neural networks

- Convolution operation (*search: “convolutional neural network explained”, “convolution operation CNN”*)
- Padding, strides, pooling (*search: “padding and strides CNN”, “max pooling CNN”*)

- LeNet-5 architecture (*search: “LeNet-5 architecture”*)
- Backpropagation in CNNs (*search: “backpropagation in CNN”*)
- Data augmentation (*search: “data augmentation deep learning”*)
- Pretrained models and transfer learning: feature extraction vs fine-tuning (*search: “transfer learning Keras”, “feature extraction vs fine tuning”*)
- Visualization of filters and feature maps (*search: “visualize CNN filters and feature maps”*)
- Functional API (*search: “Keras functional API”*)
- Project: cats vs dogs image classification (*search: “cats vs dogs classification CNN”*)

Stage 4 — Sequence models and attention

- Recurrent neural networks (RNN) (*search: “recurrent neural network explained”*)
- RNN architecture and forward propagation (*search: “RNN forward propagation architecture”*)
- Types of RNN: one-to-many, many-to-one, many-to-many (*search: “types of RNN many to many one to many”*)
- Backpropagation through time (*search: “backpropagation through time”*)
- Limitations of RNNs (*search: “problems with RNN”*)
- LSTM (*search: “LSTM explained”*)
- GRU (*search: “GRU explained”*)
- Stacked and bidirectional RNNs (*search: “stacked deep RNN”, “bidirectional LSTM”*)
- Encoder–decoder (sequence-to-sequence) models (*search: “encoder decoder seq2seq architecture”*)
- Attention mechanism (Bahdanau and Luong) (*search: “attention mechanism seq2seq”, “Bahdanau vs Luong attention”*)
- Projects: sentiment classification with an LSTM; next-word prediction (*search: “LSTM sentiment analysis Keras”, “next word prediction LSTM”*)

Stage 5 — Transformers

- Self-attention: queries, keys, values, scaled dot-product (*search: “self attention transformers explained”*)
- Multi-head attention (*search: “multi head attention explained”*)
- Positional encoding (*search: “positional encoding transformers”*)
- Layer normalization vs batch normalization (*search: “layer normalization vs batch normalization”*)
- Transformer encoder (*search: “transformer encoder architecture”*)
- Masked self-attention (*search: “masked self attention transformer”*)
- Cross-attention (*search: “cross attention transformer”*)
- Transformer decoder (*search: “transformer decoder architecture”*)
- Transformer inference and text generation (*search: “transformer inference text generation”*)
- Project: transformer implementation from scratch (*search: “build transformer from scratch”*)

Stage 6 — Large language models

- Tokenization (BPE) and embeddings (*search: “BPE tokenization explained”, “word embeddings explained”*)
- Next-token prediction objective (*search: “language model next token prediction”*)
- GPT: building a small model from scratch (*search: “build GPT from scratch”*)

- Prompting (*search: “prompt engineering tutorial”*)
- Retrieval-augmented generation (RAG) (*search: “retrieval augmented generation explained”*)
- Fine-tuning with LoRA/PEFT (*search: “LoRA fine tuning explained”*)
- Agents and tool use (*search: “LLM agents tutorial”*)
- Project: small RAG application (*search: “build RAG application”*)